

UNIVERSITE CHEIKH ANTA DIOP

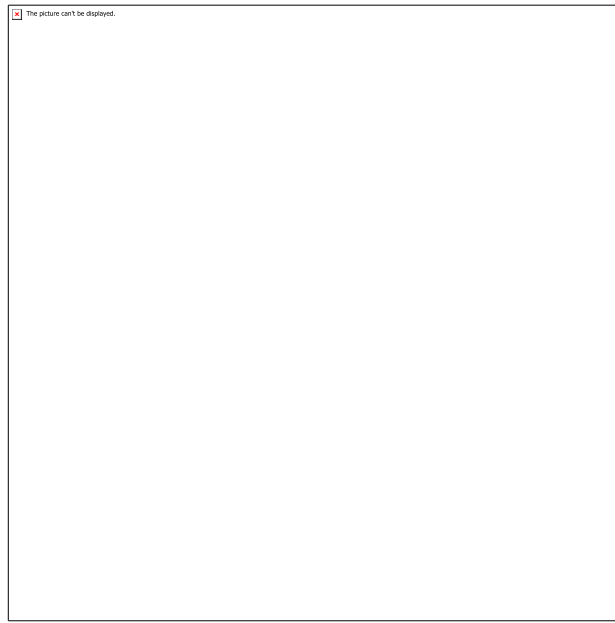


**Ecole Supérieure Polytechnique
Département Génie Informatique
Année Universitaire 2025-2026
DIT2
Technologie .NET et C#**

Etudiante

Fatou Gueye NDONG

DIT 2 Informatique



Année : 2025-2026

Professeur : M. Ousmane DIALLO

Question 1 : Définir et expliquer la différence entre .NET, .NET Platform, .NET Core

.NET : C'est le terme générique (umbrella term) qui désigne l'ensemble constitué du .NET Standard et de toutes les implémentations et charges de travail .NET. Il est toujours écrit en majuscules. C'est un framework de développement multiplateforme, open source et gratuite, permettant de créer de nombreux types d'applications. Elle repose sur un runtime haute performance utilisé en production par de nombreuses applications à grande échelle.

.NET Platform : L'expression ".NET platform" (ou plateforme .NET) est utilisée dans la documentation pour désigner soit une implémentation de .NET, soit l'ensemble de la pile .NET incluant toutes ses implémentations. Elle fait référence au système d'exploitation et au matériel sur lequel il tourne (Windows, macOS, Linux, iOS, Android).

.NET Core : C'est une implémentation multiplateforme, haute performance et open source de .NET. Elle inclut le CoreCLR (moteur d'exécution), le CoreRT (runtime AOT, en développement), la Base Class Library Core (CoreFX) et le SDK Core. .NET Core 1.0 a été publié le 27 juin 2016. C'est la réponse de Microsoft à la limitation du .NET Framework classique, qui était restreint à Windows.

Différence :

En résumé, .NET est le terme global ; .NET Platform désigne une implémentation ou la pile complète ; .NET Core est une implémentation spécifique, multiplateforme et open source.

Question 2 : Donner des éléments permettant de montrer la présence importante de l'écosystème .NET dans celui du développement logiciel en général.

Plusieurs éléments montrent l'importance de l'écosystème .NET dans le développement logiciel en général :

- Popularité du langage C# : Selon l'Index TIOBE d'octobre 2025, C# se classe 5e langage de programmation mondial avec un taux de 6,94 %.
- Performances élevées : .NET 6 traite plus de 7 millions de requêtes/seconde (ASP.NET Core), soit plus de 10x plus rapide que Node.js. Entity Framework Core a enregistré une amélioration de +92 % entre .NET 5 et .NET 6.
- Amélioration continue des performances : .NET 6.0 (+20 %), .NET 7.0 (+19 %), .NET 8.0 (+27 %), .NET 9.0 (+9 %) selon les benchmarks AIS.NET.
- Adoption massive en production : Le proxy inversé YARP 1.0 basé sur .NET traite plus de 100 milliards de requêtes/mois et 7,5 pétaoctets transférés/mois, utilisé par Microsoft Dynamics 365 et Azure App Service.
- Écosystème NuGet très riche : 797 237 311 427 téléchargements de packages, 10 437 329 versions de package et 477 408 packages uniques disponibles sur www.nuget.org.
- Grande communauté : .NET 10.0 compte 506 contributeurs et 16 777 contributions.
- Salaires attractifs : En France, un développeur .NET gagne en moyenne 43 538 €/an (28 517 390 FCFA), et aux USA 124 954 \$/an (76 282 417 FCFA).
- Couverture large des types d'applications : Cloud (Azure), Web (ASP.NET, Blazor), Desktop (.NET MAUI, WPF, WinForms), Mobile (.NET MAUI, Xamarin), Gaming (Unity), IoT (ARM32/ARM64), IA (ML.NET, .NET for Apache Spark).

Question 3 : Définir et expliquer la différence entre CLR, CoreCLR.

CLR (Common Language Runtime) : C'est le moteur d'exécution du .NET Framework classique. Il gère l'allocation et la gestion de la mémoire, et fonctionne comme une machine virtuelle qui exécute les applications tout en générant et compilant le code à la volée grâce à un compilateur JIT. L'implémentation CLR de Microsoft est réservée à Windows uniquement. Il inclut : JIT & NGEN, Garbage Collector, Security Model, Exception Handling, Loader & Binder.

CoreCLR (.NET Core Common Language Runtime) : C'est le moteur d'exécution de la plateforme .NET Core. Il est construit à partir de la même base de code que le CLR. À l'origine, CoreCLR était le runtime de Silverlight et a été conçu pour fonctionner sur plusieurs plateformes (Windows et OS X). Il est désormais intégré dans .NET Core et représente une version simplifiée du CLR. C'est toujours une machine virtuelle avec des capacités JIT et d'exécution de code, mais contrairement au CLR, il est multiplateforme et prend en charge de nombreuses distributions Linux.

Différence principale : Le CLR est limité à Windows, tandis que le CoreCLR est multiplateforme (Windows, Linux, macOS).

Question 4 : Expliquer les concepts IL, NGEN, AOT et JIT.

IL (Intermediate Language) : C'est le langage intermédiaire vers lequel les langages .NET de haut niveau (comme C#) compilent. Il s'agit d'un jeu d'instructions indépendant du matériel. L'IL est parfois appelé MSIL (Microsoft IL) ou CIL (Common IL). Le bytecode IL peut par lui-même être indépendant de l'OS et de l'architecture du CPU.

JIT (Just-In-Time compiler) : Compilateur à la volée. Il traduit l'IL en code machine compréhensible par le processeur. La compilation JIT se produit à la demande, sur la même machine que celle où le code doit s'exécuter, pendant l'exécution de l'application. Le temps de compilation fait donc partie du temps d'exécution. Un compilateur JIT connaît le matériel réel et peut libérer les développeurs de l'obligation de fournir différentes implémentations.

NGEN (Native image GENeration) : C'est une technologie que l'on peut considérer comme un compilateur JIT persistant. Elle compile habituellement le code sur la machine où il est exécuté, mais la compilation se produit généralement au moment de l'installation. Cela évite la compilation JIT à chaque lancement.

AOT (Ahead-Of-Time compiler) : Compilateur anticipé. Comme le JIT, ce compilateur traduit l'IL en code machine. Contrairement au JIT, la compilation AOT se produit avant l'exécution de l'application et est généralement effectuée sur une machine différente. Comme les chaînes d'outils AOT ne compilent pas au moment de l'exécution, elles peuvent passer plus de temps à optimiser. Comme le contexte de l'AOT est l'application entière, le compilateur AOT effectue également la liaison inter-modules et l'analyse complète du programme, produisant un seul exécutable.

Question 5 : Quels sont les avantages et les inconvénients respectifs de AOT et JIT comme technologies de compilation.

Compilation JIT (Just-In-Time)

Avantages :

- Adaptatif : connaît le matériel réel de la machine cible.
- Portable : un même binaire IL fonctionne sur toutes les plateformes.
- Optimisations dynamiques possibles selon le contexte d'exécution.
- Les développeurs n'ont pas à fournir différentes implémentations pour chaque architecture.

Inconvénients :

- Le temps de compilation fait partie du temps d'exécution, ce qui peut ralentir le démarrage.
- Le compilateur doit équilibrer le temps passé à optimiser contre les gains que le code résultant peut produire.

Compilation AOT (Ahead-Of-Time)

Avantages :

- Pas de compilation au démarrage : l'application démarre plus rapidement.
- Plus de temps disponible pour l'optimisation du code.
- Analyse complète du programme : liaison inter-modules et whole-program analysis.
- Produit un seul exécutable natif.
- Supprime le code inutilisé, réduisant la taille de l'application.

Inconvénients :

- Moins flexible : compilé pour une architecture cible spécifique.
- La compilation se produit généralement sur une machine différente, ce qui complexifie le processus de build.
- Moins d'adaptabilité aux optimisations dynamiques basées sur le contexte d'exécution réel.

Question 6 : Que signifient RyuJIT ? Roslyn ?

RyuJIT : RyuJIT est le compilateur JIT (Just-In-Time) utilisé dans .NET. Il compile le code IL (Intermediate Language) en code natif/machine compréhensible par le processeur. C'est le compilateur JIT par défaut intégré dans le CLR et le CoreCLR de .NET moderne.

Roslyn : Roslyn est le compilateur open source intégré dans Visual Studio. Il permet un développement plus facile d'outils liés au code source. Il fournit un SDK permettant d'écrire ses propres refactorisations et analyses de code. Roslyn est membre du projet de la .NET Foundation. Il compile les langages C# et VB.NET et expose des APIs pour analyser, générer et transformer du code.

Question 7 : Qu'est-ce que c'est un Assembly ? un Library ? un Package ?

Assembly : Un Assembly est un fichier .dll ou .exe qui peut contenir une collection d'APIs pouvant être appelées par des applications ou d'autres assemblies. Un assembly peut inclure des types tels que des interfaces, des classes, des structures, des énumérations et des délégués. Les assemblies dans le dossier bin d'un projet sont parfois appelés des binaires.

Library (Bibliothèque) : Une Library est une collection d'APIs pouvant être appelées par des applications ou d'autres bibliothèques. Une bibliothèque .NET est composée d'un ou plusieurs assemblies. C'est un ensemble de fonctionnalités réutilisables regroupées pour faciliter le développement.

Package : Un package NuGet (ou simplement package) est un fichier .zip contenant un ou plusieurs assemblies portant le même nom, accompagnés de métadonnées supplémentaires (comme le nom de l'auteur). Le fichier .zip a une extension .nupkg et peut contenir des ressources (.dll, .xml) pour plusieurs frameworks et versions cibles. Les ressources définissant l'interface se trouvent dans le dossier ref, et celles définissant l'implémentation se trouvent dans le dossier lib.

Question 8 : Qu'est-ce que Nuget et quelle est son utilité.

NuGet (www.nuget.org) est le gestionnaire de packages officiel pour l'écosystème .NET.

Son utilité :

- Fournit des packages pour faciliter la création d'applications .NET.
- Permet des mises à jour très fréquentes des bibliothèques.
- Est distribué sous forme d'extension des environnements de développement : Visual Studio, JetBrains Rider, SharpDevelop et Visual Studio Code.
- Après avoir installé un package NuGet, on peut y faire référence dans le code avec l'instruction `using <namespace>`, puis appeler le package via son API.

Quelques chiffres témoignant de son importance :

- 797 237 311 427 téléchargements de packages
- 10 437 329 versions de package
- 477 408 packages uniques

Question 9 : Quels sont les objectifs de la plateforme MONO ? **Comment se positionne-t-elle aujourd'hui dans l'environnement .NET**

Objectifs de Mono : Mono est une implémentation open source et multiplateforme de .NET. Son objectif principal est de permettre l'exécution d'applications .NET sur des systèmes autres que Windows (Linux, macOS). Il a son propre Runtime, ses propres compilateurs et sa propre BCL (Base Class Library). Un programme écrit sur Mono peut fonctionner sur Linux, Mac OS et Windows. Le CLR peut exécuter des applications Mono.

Positionnement actuel dans l'environnement .NET :

- Mono est principalement utilisé quand un runtime léger est requis (small footprint).
- C'est le runtime qui alimente les applications Xamarin sur Android, Mac, iOS, tvOS et watchOS.
- Il supporte toutes les versions publiées de .NET Standard.
- Historiquement, Mono a implémenté la grande API du .NET Framework et émulé certaines de ses capacités sur Unix.
- Mono utilise généralement un compilateur JIT, mais dispose aussi d'un compilateur statique complet (AOT) utilisé sur des plateformes comme iOS.
- Limitations : il ne fournit pas la pleine puissance du .NET classique, petite communauté, et son compilateur C# n'émet pas de code aussi optimisé que le compilateur .NET C# classique.
- Avec l'avènement de .NET Core puis .NET 5+, Mono reste surtout pertinent dans le contexte mobile via Xamarin/.NET MAUI.

Question 10 : À quoi sert Xamarin ? Quels sont les os qu'il cible ? Quels sont ses liens avec la plateforme Mono ?

À quoi sert Xamarin : Xamarin est un framework permettant de développer des applications mobiles multiplateformes en C# et .NET. Il permet d'écrire une logique métier partagée en C# tout en accédant aux APIs natives de chaque plateforme mobile.

Systèmes d'exploitation ciblés :

- Android
- iOS
- macOS
- tvOS
- watchOS

Liens avec la plateforme Mono : Xamarin est directement basé sur Mono. Le runtime Mono est celui qui alimente les applications Xamarin sur Android, Mac, iOS, tvOS et watchOS. Xamarin utilise le runtime Mono pour exécuter le code C# sur ces plateformes. Sur iOS par exemple, Mono est utilisé avec la compilation AOT (Ahead-Of-Time) puisque la compilation JIT n'est pas autorisée par Apple. Aujourd'hui, Xamarin a évolué vers .NET MAUI (.NET Multi-platform App UI) qui unifie Xamarin.Android et Xamarin.iOS sous une seule base de code .NET.

Question 11 : Quelle est la signification de l'acronyme UWP ? Lister les langages utilisés et les différents types d'appareil ciblés par UWP.

Signification : UWP signifie Universal Windows Platform (Plateforme Windows Universelle). C'est une implémentation de .NET utilisée pour créer des applications Windows modernes, tactiles, et des logiciels pour l'Internet des Objets (IoT).

Langages utilisés :

- C++
- C#
- VB.NET
- JavaScript

Note : Lorsque C# et VB.NET sont utilisés, les APIs .NET sont fournies par .NET Core.

Types d'appareils ciblés :

- PCs
- Tablettes
- Phablets (smartphone-tablette)
- Téléphones
- Xbox
- Appareils IoT

UWP fournit des services comme un app store centralisé, un environnement d'exécution (AppContainer) et un ensemble d'APIs Windows (WinRT) à utiliser à la place de Win32.

**Question 12 : Quelle est la signification de l'acronyme MAUI ?
Quels développements peut-on faire avec ?**

Signification : MAUI signifie .NET Multi-platform App UI (.NET Interface Utilisateur Multi-plateforme). C'est une infrastructure multiplateforme permettant de créer des applications mobiles et de bureau natives en C# et XAML.

Développements possibles avec .NET MAUI :

- Applications Android
- Applications iOS
- Applications macOS
- Applications Windows

Tout cela à partir d'une seule base de code partagée, suivant le principe du développeur "write-once run-anywhere" (écrire une fois, exécuter partout), tout en offrant un accès approfondi à tous les aspects de chaque plate-forme native.

Question 13 : Citez des avantages de l'utilisation de Blazor ?

Blazor est une infrastructure permettant de créer des interfaces utilisateur web côté client interactives avec .NET. Ses avantages sont :

- Développement d'UIs interactifs enrichis à l'aide de C# (pas besoin de JavaScript).
- Interface utilisateur en HTML et CSS avec large prise en charge des navigateurs, y compris les navigateurs mobiles.
- Possibilité de créer des applications de bureau et mobiles hybrides avec .NET et Blazor.
- Compatible avec les plateformes d'hébergement modernes comme Docker.
- Réutilisation des connaissances C# et .NET pour le développement frontend.
- Partage du code entre la logique serveur et la logique client.

Question 14 : Quelles sont les prochaines étapes de la feuille de route Microsoft concernant .NET ?

D'après le cours, la feuille de route Microsoft pour .NET se présente comme suit :

- .NET 6 (nov. 2021) — LTS (Long Term Support) : support de 36 mois.
- .NET 7 (nov. 2022) — STS (Standard Term Support) : support de 18 mois.
- .NET 8 (nov. 2023) — LTS : support de 36 mois. Dernière version LTS en cours (Latest release).
- .NET 9 (nov. 2024) — STS : version courante à Support Standard.
- .NET 10 (nov. 2025) — LTS : prochaine version à Long Term Support, en cours de développement.

Les tâches asynchrones prévues comprennent également :

- La certification C# de base avec Microsoft.
- L'utilisation de Blazor pour les interfaces web.
- La création d'applications mobiles avec .NET MAUI.

Microsoft alterne ainsi entre versions LTS (support 36 mois) et STS (support 18 mois), avec une version majeure par an chaque mois de novembre.

Merci